

Nine chores for three agents

A complete EFX existence argument and its exact computational certificate

Anonymous

8 October 2026

Abstract

Every instance of three agents and nine indivisible chores with nonnegative additive real costs admits a complete envy-free-up-to-any-chores (EFX) allocation. The deletion quantifier includes owned zero-cost chores. We reduce a hypothetical counterexample to a canonical domain with 21 real variables and encode failure of all allocations. A finite refutation is checked by separate exact software, using rational weighted-sum contradictions and reverse unit propagation. The written reduction is assessed separately; the argument is not formalised end to end in a proof assistant, and internal AI-assisted review is not external peer review. We also give an exhaustive rational-input allocation finder, a general row-minimum-lowering lemma, and a supplementary finite representative bound. The result concerns nine chores, not an arbitrary number of chores or empirical burden measurement.

Verification terminology. A separate exact program checks the computational refutation; neither a solver verdict nor receipt composition alone establishes it. “Independently checked” means checking separate from the producer, not a second independently implemented verifier or external human refereeing. UNSAT means unsatisfiable: no assignment satisfies the encoded constraints. SMT abbreviates satisfiability modulo theories.

1 Result and scope

Theorem 1 (Nine-chores EFX existence). Let M be a set of nine indivisible chores and let $N = \{0, 1, 2\}$ be three agents. For each agent i , let $c_i : M \rightarrow \mathbb{R}_{\geq 0}$ be an arbitrary nonnegative cost function, extended additively to subsets of M . There exists a complete labelled allocation $A = (A_0, A_1, A_2)$, whose bundles partition M , such that

$$c_i(A_i \setminus \{g\}) \leq c_i(A_j) \quad \text{for every } i \neq j \text{ and every } g \in A_i.$$

Empty bundles are allowed. The quantifier includes an owned chore with zero cost. No bound, integrality assumption, common ranking, or common cost scale is imposed on the original costs. Both sides of each comparison are evaluated by the same agent.

The proof has a written reduction and a finite computational part. Every hypothetical counterexample can be transformed into a model of one explicitly specified formula over 21 real variables. The formula contains the failure condition for every allocation into three nonempty bundles; the other allocations fail by a separate argument. An exact refutation of this formula establishes the theorem. The final computational claim is governed by the verification status above and the certificate records in Section 4.

The proof package includes a separate allocation finder. For a supplied rational 3×9 matrix, it checks at most $3^9 = 19,683$ allocations and returns an allocation with all 18 original, exact

deletion inequalities. This makes the conclusion usable without rerunning the universal proof search.

1.1 Relation to prior work

Zhang’s inspected version 2, dated 6 October 2026, establishes eight-chore existence in Theorem 1.2; Section 6 identifies nine chores as the next case [1]. Kobayashi, Mahara and Sakamoto supply the smaller baseline (Theorem 3.1) and the insertion fact used here (Lemma 4.2, full-preprint numbering) [2]. The present proof uses ordinary eight-chore existence, not a prescribed-agent strengthening.

Component	Contribution boundary
Eight-chore existence and shared-cheapest insertion	Prior results [1,2]; assumptions checked against the stated additive model
Nine-chore existence	Present written reduction and exact finite refutation
Minimum lowering	Elementary lemma stated here for any finite number of agents and chores; no absolute priority claim
Finite representative bound	Supplementary specialization of classical determinant methods [4], not needed for the main certificate
Exact checking and allocation finder	Reproducible implementation, not a claim to invent computer-assisted fair division

He and Tao give counterexamples for every number of agents at least four [5]. They do not settle the present three-agent case. Computational fair-division precedents include Alkassar, Fouz and Mehlhorn’s additive-goods result [6] and Akrami and coauthors’ SAT-based work for nonadditive goods [7]. Goods and chores use different deletion comparisons. In [7], the Lean validation concerns the abstract encoding, not end-to-end verification of the search implementation. Our exact checker likewise does not formalize the written reduction.

The contribution claimed here is the stated nine-chore result and its checkable evidence. A bounded publication-intake search on 8 October 2026 does not establish absolute historical priority. This remains an unrefereed candidate, without external peer-review endorsement.

2 From arbitrary costs to a canonical counterexample

For a nonempty bundle S , define its maximum deletion residual for agent i by

$$r_i(S) = c_i(S) - \min_{g \in S} c_i(g), \quad r_i(\emptyset) = 0.$$

This equals the largest cost remaining after deletion of one owned chore. Nonnegativity therefore makes EFX equivalent to $r_i(A_i) \leq c_i(A_j)$ for every $i \neq j$. If a cheapest owned chore costs zero, deleting that chore leaves the entire owned cost; this case is included throughout.

2.1 Strict positive counterexamples suffice

Suppose a nonnegative matrix C has no EFX allocation. For each of the finitely many complete allocations A , choose one failed comparison. Its failure is a strict linear inequality

$$F_{A,i,j,g}(C) = \sum_{h \in A_i \setminus \{g\}} c_{ih} - \sum_{h \in A_j} c_{ih} > 0.$$

Each selected linear form is positive at C . Their simultaneous strict positivity defines an open neighborhood of C . A sufficiently small perturbation into the positive orthant preserves all these failures. Avoiding the finitely many rowwise equality hyperplanes additionally makes the nine entries within each row distinct. Thus, if a counterexample exists, a strictly positive counterexample with distinct costs in each row exists.

This argument does not assume positivity of the original input. It uses the strictness of failure and the finiteness of the allocation set to preserve a witness for every allocation at once.

2.2 A chore cheapest for two agents can be inserted

Lemma 2 (Shared-cheapest insertion; [2], Lemma 4.2). Suppose e is a globally cheapest chore for agents 1 and 2, while agent 0 is unrestricted. If the other chores admit an EFX allocation, then all chores, including e , admit an EFX allocation.

Proof. Start with an EFX allocation (B_0, B_1, B_2) of the old chores. From each agent $i \in \{1, 2\}$ draw an arc to the owner of an old bundle of minimum cost according to i . Allow a self-loop. Agent 0 has no outgoing arc.

If this graph contains a directed cycle, rotate the bundles along that cycle. Every changed agent receives one of its minimum-cost old bundles and is therefore ordinarily envy-free. Add e to the new bundle B of one cycle agent i . Since e is cheapest for i ,

$$r_i(B \cup \{e\}) = c_i(B).$$

That value is at most every old bundle total according to i , so the recipient is EFX. The identity also holds for an empty B . Other changed agents received minimum-cost bundles, and unchanged agents retain their own bundles. Increasing one comparison bundle cannot create new envy for another agent. The result is EFX.

If there is no directed cycle, every path ends at agent 0. First add e to B_0 . If agent 0 remains EFX, no other agent is harmed and the construction is complete. Otherwise let B_j , with $j \neq 0$, be a minimum-cost bundle for agent 0 among the augmented collection. Such an unchanged minimum bundle exists: failure of EFX means the maximum deletion residual of the augmented owned bundle exceeds another bundle's total, so that owned bundle cannot itself be minimum.

Follow the path $j = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_t = 0$. Give B_j to agent 0. For $0 \leq s < t - 1$, give old $B_{v_{s+1}}$ to agent v_s . Give $B_0 \cup \{e\}$ to agent v_{t-1} . Agent 0 receives a minimum-cost bundle in the augmented collection. Each intermediate agent receives a minimum-cost old bundle, which remains minimum when another bundle grows. The last agent regards old B_0 as minimum, and its augmented residual is exactly $c_{v_{t-1}}(B_0)$. These agents are all EFX. Agents outside the path keep their bundles and face the same old bundle collection with only one bundle enlarged. They retain EFX. This also covers ties and empty old bundles. $\square$

This is the three-agent form of the insertion fact used in [2]. The proof above needs only that the inserted chore is cheapest for the two specified agents; it assumes no agreement on their rankings of the other chores.

Apply the lemma to a positive, rowwise distinct nine-chore counterexample. If two agents shared their cheapest chore, delete it, apply ordinary eight-chore EFX existence, and insert it again. This would contradict nonexistence. Hence a generic counterexample has three distinct unique minimum chores. This step is applied while costs are still strictly positive.

2.3 Lower one minimum entry in each row to zero

Lemma 3 (Minimum lowering). For any finite number of agents and any finite nonempty set of chores with nonnegative additive costs, choose a globally cheapest chore e_i for each agent

i . Change only $c_i(e_i)$ to zero, leaving the other entries unchanged, independently for every row. Let D denote the resulting matrix. Then

$$\text{EFX}(D) \subseteq \text{EFX}(C).$$

Proof. Fix an evaluating row and an owned bundle S . If $e_i \notin S$, its residual is unchanged. If $e_i \in S$, its total cost and its minimum cost both decrease by $c_i(e_i)$, so its residual again remains unchanged. Every comparison-bundle total weakly decreases. Therefore an inequality asserting that an owned residual is at most a comparison total under D also holds under C . Apply this to every agent and comparison. $\square$

A counterexample consequently remains a counterexample after lowering one minimum per row to zero. Starting from the generic matrix above, there is now exactly one zero in each row, at three distinct chore positions, and every other entry is positive. The operation changes one entry in a row; it is not uniform subtraction from the whole row.

2.4 Relabel and scale

Relabel the distinct minimum chores so that agent i has zero at chore i . Choose an agent to carry label 0. If necessary, swap the other two agents together with their minimum-chore labels to obtain $c_{01} < c_{02}$. Sort the six remaining chore columns in decreasing order of agent 0's costs. All these relabellings preserve existence by a bijection of complete allocations.

The three reference costs c_{01}, c_{10}, c_{20} are positive. Divide the corresponding rows by these numbers. Independent positive scaling of a row preserves every EFX comparison in that row. We obtain

$$c_{00} = c_{11} = c_{22} = 0, \quad c_{01} = c_{10} = c_{20} = 1,$$

$$c_{02} > 1, \quad c_{03} > c_{04} > c_{05} > c_{06} > c_{07} > c_{08} > 0.$$

All other free entries are strictly positive. Six matrix entries are fixed constants, leaving 21 real variables. The serialized domain has 21 positivity assertions, the assertion $c_{02} > 1$, and five successive free-column comparisons: 27 domain assertions in total. There is no cost ceiling, integrality condition, uniform failure margin, cross-row total comparison, or prescribed envy-free agent in this domain.

3 The complete counterexample formula

3.1 Allocations with an empty bundle

There are $3^9 = 19,683$ labelled assignments. By inclusion-exclusion,

$$3^9 - 3 \cdot 2^9 + 3 = 18,150$$

assignments give every agent a nonempty bundle. The remaining 1,533 assignments have an empty bundle.

In the canonical domain, every row has exactly one zero. If some bundle is empty, another owner has at least two chores. Choose a positive-cost chore in that owned bundle and remove a different owned chore. The remaining cost is positive, while the empty comparison bundle costs zero. Thus the allocation fails literal EFX.

This argument is confined to the actual canonical domain. For an arbitrary all-zero matrix, empty-bundle allocations can indeed be EFX. The package explicitly enumerates a valid combinatorial witness for each of the 1,533 omitted assignments.

3.2 One failure clause for every other allocation

Let $\mathcal{S}$ be the 18,150 surjective assignments. For each $A \in \mathcal{S}$, write

$$\text{Bad}_A(C) = \bigvee_{\substack{i \neq j \\ g \in A_i}} \left(\sum_{h \in A_i \setminus \{g\}} c_{ih} - \sum_{h \in A_j} c_{ih} > 0 \right).$$

This is exactly the failure of EFX for A . Conjoin every such clause with the canonical domain predicate $\mathcal{D}$ (distinct from the lowered cost matrix D):

$$F_9(C) = \mathcal{D}(C) \wedge \bigwedge_{A \in \mathcal{S}} \text{Bad}_A(C).$$

The serialized formula omits only deletion literals that are redundant under the imposed partial order. For an owned bundle, removing a cheapest owned chore gives the largest residual. Every chore that could be cheapest under the imposed order is retained. An actual cheapest chore cannot be excluded by a valid strictly cheaper comparison. Conversely, every retained literal is an original EFX failure. Bundles of size at most one contribute no possible failure, because their residual is zero. In particular, deleting an owned pinned-zero chore is retained.

An independent standard-library audit reconstructs every declaration, fixed substitution, domain assertion and allocation clause from the economics definition. It checks 191,104 retained literal positions, including 34,776 positions deleting an owned pinned zero. The audit uses exact integer affine coefficients and complete allocation enumeration.

Quantity	Exact value
Original complete labelled allocations	19,683
Surjective allocation clauses	18,150
Omitted empty-bundle assignments covered by proof	1,533
Free real variables	21
Domain assertions	27
Total original assertions	18,177
Retained literal positions	191,104
Retained pinned-zero deletion positions	34,776

The decisive input is `continuation/exact_search/zero_minimum9_reference_strict.smt2`, an 11,182,371-byte file with SHA-256:

65b0e3d6b2f234a6391db058771b8597a7bdd6feb003813ddc88630843ceab5b

The preceding lemmas prove that every original counterexample would yield a real model of this exact formula. Conversely, a model of this formula would give a canonical counterexample: the clauses exclude every surjective allocation, and the empty-bundle argument excludes all the others. Unsatisfiability therefore decides the complete target over the unbounded real cost domain.

4 Exact computational refutation

4.1 Search result and recorded trace

The frozen input returned UNSAT in two Z3 runs. The first, without native proof retention, completed its check in 591.670 seconds. A second run using a clause recorder returned UNSAT in 1,327.735 seconds and retained the information needed for an independently checked refutation. These are observed check times in the recorded environment, not runtime guarantees.

The second run captured 18,066 exact affine atoms, 18,178 input records, 3,234,215 unique arithmetic theory clauses, and 2,480,762 ordered Boolean RUP records. The extra input record is the truth unit; it is not an additional economics premise. Every captured file was sealed, counted, and hash-checked after normal completion. The original SMT input and all original affine atoms were then reconstructed independently.

Z3 supplies untrusted proof material through its clause callback interface [3]. Its UNSAT label is not used as a premise by the independent replay. Callback assumption clauses are also not admitted as new axioms.

4.2 Arithmetic axioms by exact weighted sums

Every nonconstant atom is represented by a primitive integer affine form

$$q(x) = a \cdot x + b \leq 0.$$

An arithmetic theory clause is a disjunction of signed atoms. To validate it, negate all its literals and exhibit a contradiction among the resulting affine inequalities. Each negated literal has the form $h_k(x) \leq 0$ or $h_k(x) < 0$.

Its certificate lists nonnegative rational weights λ_k satisfying

$$\sum_k \lambda_k a_k = 0, \quad K = \sum_k \lambda_k b_k.$$

If $K > 0$, summing the premises would imply $K \leq 0$, a contradiction. If $K = 0$ and a strict premise has positive weight, summing would imply $0 < 0$, again a contradiction. These are the only accepted contradiction cases. The checker performs all operations with exact integers and rational fractions, rejects floating-point coefficients or multipliers, and verifies the exact clause and selected index to which each certificate belongs.

Numerical linear programming may help the untrusted producer find a candidate multiplier vector. It supplies no premise of the final proof: every accepted identity and sign is recomputed exactly by a separate standard-library checker.

4.3 Boolean deductions by RUP

RUP means reverse unit propagation. To check a derived clause $L_1 \vee \dots \vee L_t$, temporarily assume that every L_k is false. The certificate lists previously admitted clauses in a propagation order. Each listed clause must either force its only unassigned literal or, at the final step, be entirely false and yield a contradiction. Therefore the derived clause follows from earlier clauses.

The checker requires every reason to be an earlier known clause, verifies that each forcing step is truly unit, rejects satisfied or circular reasons, and requires a final derived empty clause. The empty clause expresses contradiction with no temporary goal assumptions. A backward dependency check ensures that the retained trace contains exactly the clauses used by the final contradiction.

Original axioms are bound to their reconstructed input clauses. Theory axioms must be in the selected list and have complete exact arithmetic certificates. No callback assumption or solver

verdict is an admissible proof axiom. Hence induction through the checked derivation proves that the original formula has no real model.

4.4 Final certificate statistics

The complete independent replay passed original-input reconstruction, the final Boolean contradiction, and exact arithmetic validation of every admitted theory axiom. The following counts describe the proof actually checked.

Checked proof component	Eight-chore baseline	Nine-chore target
Selected original clauses	898	4,550
Selected exact arithmetic axioms	27,637	370,780
Derived RUP clauses	10,845	172,146
Ordered propagation reasons	405,499	8,088,323

The completed eight- and nine-chore receipts are linked from the archive’s AI index. Proof identities, source hashes, selected-clause coverage and stage receipts are retained there. Original input reconstruction, Boolean refutation and complete arithmetic coverage are separate required checks; a partial pass establishes no complete UNSAT certificate.

The trusted mathematical boundary consists of the stated reductions, the cited smaller-case baseline, the exact checking calculus and the implementation/runtime used to replay it. Z3, the C++ trace producer and the numerical multiplier search need not be trusted for the correctness of a successfully replayed proof. This is a computer-assisted proof with an independent exact checker; it is not an end-to-end formalization in a proof assistant.

5 Completion of the existence argument

Suppose the theorem were false. Section 2 produces a positive generic counterexample, rules out shared cheapest chores, lowers the three distinct row minima to zero, and puts the matrix in the 21-variable canonical domain. Section 3 proves that the matrix satisfies every assertion of F_9 . Section 4 supplies the checked refutation of that exact formula. This is a contradiction. Therefore every instance in the theorem admits a complete EFX allocation. $\square$

The argument uses ordinary eight-chore EFX existence only in the shared-minimum insertion step. Besides the cited theorem [1], the package provides a completed eight-chore residual certificate, its full input audit, and witnesses for all 765 omitted empty-bundle assignments. That case reduces shared minima to ordinary seven-chore existence. The retained seven-chore CPC proof was checked by Ethos against the exact reference input, with final false required at global scope. Its shared-minimum step uses the cited ordinary six-chore theorem of [2].

Thus the alternative dependency chain is the cited six-chore theorem, the independently checked seven-chore residual, the independently checked eight-chore residual, and the independently checked nine-chore residual, with the written genericity and insertion reductions between them. The preserved seven-chore proof has SHA-256 62d8754d1e0f592a8c28b39d64de02d9579287d05b2346ff6e6349a6cb02961c. No prescribed-agent strengthening is used.

6 Finding and checking an allocation

For rational input costs, the following algorithm is complete.

1. Multiply each row by the least common multiple of its denominators. This produces nonnegative integers and preserves the EFX comparisons.

2. For each row and every subset of the nine chores, precompute its total cost and its maximum deletion residual, setting the empty residual to zero.
3. Enumerate the 19,683 labelled assignments. Accept the first for which the three owned residuals satisfy all six comparisons with the other bundles.
4. Re-evaluate every original owned-chore deletion inequality using exact rational input costs. Return the allocation and the resulting certificate.

The theorem ensures that the enumeration succeeds. The finder also checks its answer directly, so verification of a returned allocation is independent of the universal proof. The implementation uses only Python’s standard library, and accepts integer costs, exact JSON decimal numbers, or fraction strings such as "2/3".

From the extracted package root:

```
python3 src/solve_efx9.py examples/costs9.json --out allocation.json
```

The output numbers agents and chores from 1. It gives the bundle of every agent, the owner of every chore, and all 18 deletion comparisons with exact nonnegative slacks. Empty bundles and zero-cost deletions are supported directly.

As a worked example, consider the following costs, with chores numbered 1 to 9:

Agent	1	2	3	4	5	6	7	8	9
1	0	1	2	5	5	3	3	2	1
2	1	0	2	3	3	5	5	1	2
3	1	2	0	5	3	1	1	3	5

The finder returns $A_1 = \{1, 2, 3, 8, 9\}$, $A_2 = \{4, 5\}$, and $A_3 = \{6, 7\}$. Agent 1’s owned total is 6, its minimum is zero, and its residual is 6; the other bundles cost it 10 and 6. Agent 2’s owned residual is 3, and it evaluates the other bundles at 6 and 10. Agent 3’s owned residual is 1, and it evaluates the other bundles at 11 and 8. All six residual inequalities hold, including the equality produced by agent 1 deleting its zero-cost chore.

With cached subset sums, the enumeration uses at most 19,683 groups of six integer comparisons, together with assignment bookkeeping. This counts comparisons, not unit-cost operations on arbitrary-precision integers: runtime also depends on the bit lengths of the rational input and row denominators. This is a fixed-size algorithm, not a polynomial-time algorithm for unrestricted numbers of chores. Arbitrary real inputs require exact arithmetic or comparison access; the supplied implementation handles rational data.

7 Reproducing the universal certificate

The complete nine-chore replay command is:

```
python3 continuation/verification/run_lra_with_budget.py \
--stage all \
--memory-mib 1600 \
--input continuation/exact_search/zero_minimum9_reference_strict.smt2 \
--expected-sha256 65b0e3d6b2f234a6391db058771b8597a7bdd6feb003813ddc88630843ceab5b \
--capture continuation/exact_search/zero9_minimal_replay_capture1/capture \
--trace continuation/exact_search/zero9_rup_source_cache_run1/trimmed_trace.jsonl \
--certificates continuation/exact_search/zero9_selected_farkas_final1/arithmetic_certificates.jsonl \
--selected-indices continuation/exact_search/zero9_rup_source_cache_run1/selected_theory_indices.json \
--out-directory replay_results/zero9_fresh
```

The replay driver and checking modules use only Python’s standard library. They reconstruct the original SMT-to-clause mapping and exact atoms, verify the Boolean refutation, and verify one exact arithmetic certificate for every theory axiom used. The command requests a 1,600 MiB address-space allowance. Linux enforces it; on macOS the portable driver

records that this limit is not enforced, because the corresponding system call can reject the request before checking begins. The mathematical checking functions are unchanged. A fresh output directory preserves the original receipts. The expected complete status is `PASS_INDEPENDENT_EXACT_LRA_UNSAT_CERTIFICATE`.

Receipt binding is not fresh replay. The completion binder only checks that existing stage receipts refer to compatible inputs, proof objects and coverage. It does not execute the proof checks. Its historical status string contains the word `PASS`, but a binder-only run establishes receipt consistency, not a new refutation. Fresh checking requires the three-stage command above and its newly generated stage records.

The distributed replay capture preserves the exact input records, affine atoms, and complete theory-clause source, with a separately checked source binding. The final RUP trace supplies the Boolean proof directly. Provenance records the original full solver capture; raw callback RUP and assumption logs are unnecessary for the accepted replay. The archive inventories omitted historical streams and intermediate checkpoints by hash.

The archive README gives the ordinary eight-chore replay command and the separate CPC/Ethos instructions for the optional seven/six baseline chain. Replaying the final eight- and nine-chore LRA certificates requires no Z3, cvc5, SciPy, producer executable, or compiler. Reproducing proof discovery is a different task and uses the recorded producer versions and resource limits.

File manifests identify the exact retained bytes. Two earlier eight-chore outputs were rejected because record counts or hashes exposed truncated suffixes. The accepted eight-chore objects were separately reconstructed or completed, sealed, and fully checked; their provenance and rejected originals are preserved. The nine-chore capture was sealed and independently checked complete before extraction. No failed export, interrupted run or incomplete prefix is counted as a proof.

Earlier finite-bound and regional searches remain in the historical research record. Their time-outs and partial coverage are not premises of the final nine-chore theorem. In particular, an exact two-row model shows that one earlier finite regional family was incomplete. It does not supply a third row or a counterexample to the theorem.

8 Additional mathematical result: a finite representative bound

Proposition 4 (Finite representative bound). For three agents and $m \geq 4$ chores, if a nonnegative additive EFX counterexample exists, a counterexample exists with one zero in each row and all other costs positive integers at most

$$Q_m = \left\lfloor \sqrt{(m-1)(m-2)^{m-2}} \right\rfloor.$$

In particular, $Q_8 = 571$ and $Q_9 = 2,566$. A positive representative can instead have common row minimum one and even other costs at most $2Q_m$, giving 5,132 for nine chores. The bound permits arbitrary zero columns and uses classical determinant methods [4]; no priority claim is made for this specialization.

Here is the argument. Start with a generic zero-minimum counterexample and choose one strict failed comparison for each surjective allocation. Group the failures by their evaluating row. Set $d = m - 1$ and remove the fixed zero coordinate. Each selected row vector has entries in $\{-1, 0, 1\}$ and support at most

$$|A_i| - 1 + |A_j| = m - |A_k| - 1 \leq m - 2 = d - 1.$$

Add positivity and the chosen strict order of each row's nonminimum entries. The resulting homogeneous system $Ax > 0$ can be scaled to $Ax \geq \mathbf{1}$. Positivity bounds each coordinate below by one. Minimizing the coordinate sum gives a nonempty compact minimizing face, hence a vertex. At a vertex choose d independent active constraints, giving a nonsingular integer matrix D with $Dv = \mathbf{1}$.

Write $\Delta = \det D$ and $N_j = \det D_j$, where D_j replaces column j by ones. Let g be the greatest common divisor of $|\Delta|, N_1, \dots, N_d$. The vector

$$w_j = \frac{\text{sgn}(\Delta)N_j}{g}$$

is a positive integer vector and satisfies $Aw \geq \mathbf{1}$, since $w = (|\Delta|/g)v$ and $|\Delta|/g$ is a positive integer.

Fix j and let f be the number of full-support rows in D_j . Such rows originated from rows of D with a zero in column j and ± 1 elsewhere. In the augmented matrix $[D \mid \mathbf{1}]$, these f rows have identical parity patterns. Subtracting one from each of the other $f - 1$ rows shows that every maximal minor, hence their common divisor g , is divisible by 2^{f-1} when $f \geq 1$.

Hadamard's inequality gives $|N_j|^2 \leq d^f(d-1)^{d-f}$. For $f \geq 1$,

$$\left| \frac{N_j}{g} \right|^2 \leq \frac{d^f(d-1)^{d-f}}{4^{f-1}} \leq d(d-1)^{d-1},$$

because the bound decreases as f increases. When $f = 0$, the bound $(d-1)^d$ is smaller. Thus $1 \leq w_j \leq Q_m$. Apply the construction to each row. Every selected failure persists; the empty-bundle argument covers the other allocations.

Raise each zero to $1/2$. Owned worst residuals stay unchanged, while comparison totals increase by at most $1/2$, preserving every integer failure margin of at least one. Doubling gives minimum one and other costs at most $2Q_m$.

9 References

1. Zhang, X. (2026). *EFX Allocations for Three Agents and Seven or Eight Chores*. Version 2, 6 October 2026. arXiv:2609.10585. <https://arxiv.org/abs/2609.10585v2>.
2. Kobayashi, Y., Mahara, R., and Sakamoto, S. (2025). *EFX Allocations for Indivisible Chores: Matching-Based Approach*. Theoretical Computer Science 1026, 115010. <https://doi.org/10.1016/j.tcs.2024.115010>. Inspected full preprint: <https://arxiv.org/abs/2305.04168>.
3. Z3 contributors. *Proof Logs*. Online Z3 Guide. Clause callback and proof logging documentation, inspected 7 October 2026. <https://microsoft.github.io/z3guide/programming/Proof%20Logs/>.
4. von zur Gathen, J., and Sieveking, M. (1978). *A Bound on Solutions of Linear Integer Equalities and Inequalities*. Proceedings of the American Mathematical Society 72(1), 155-158. <https://doi.org/10.1090/S0002-9939-1978-0500555-0>.
5. He, W., and Tao, B. (2026). *EFX for Additive Chores: Nonexistence, Pareto Incompatibility, and Bi-Valued Existence*. arXiv:2606.08872, Theorem 1. <https://arxiv.org/abs/2606.08872>.
6. Alkassar, E., Fouz, M., and Mehlhorn, K. (2026). *Complete EFX Allocations Exist for Four Additive Agents and Up to Nine Goods*. arXiv:2608.08590. <https://arxiv.org/abs/2608.08590>.

7. Akrami, H., Mayorov, A., Mehlhorn, K., Srinivas, S., and Weidenbach, C. (2026). *A Counterexample to EFX: $n \geq 3$ Agents, $m \geq n + 5$ Items, Submodular Valuations via SAT-Solving.* arXiv:2604.18216, version 3, Section 9. <https://arxiv.org/abs/2604.18216v3>.